Serial Manipulator Kinematics and Dynamics

This example derives the complete kinematic and dynamic model of a 2-DOF planar elbow manipulator entirely in closed form using Denavit-Hartenberg parameters, deploys a computed-torque controller, and checks the generated functions in simulation - all from first principles using Symbolic Math Toolbox™.

Serial manipulators are inherently symbolic objects: homogeneous transformation matrices parameterized by joint angles, velocity Jacobians that are literal Jacobians in the calculus sense, and Lagrangian dynamics that produce torque equations as explicit functions of configuration. The symbolic approach exposes the structure of these equations - which inertial couplings dominate, how gravity torques factor, and where singularities arise - in a way that numeric solvers never reveal.

Requires: Symbolic Math Toolbox; Robotics System Toolbox™ for the optional cross-check in Section 8.1

1. DH Forward Kinematics

1.1 Declare Symbolic Parameters

All link parameters remain symbolic so that the kinematics, Jacobian, and dynamics emerge as formulas. We substitute representative parameters only for numeric checks. A 2-DOF planar arm is the minimum system that exhibits full dynamic coupling (Coriolis, centrifugal, and inertia matrix configuration-dependence) while keeping expressions inspectable.

```
syms q1 q2 real                    % joint angles [rad]
syms dq1 dq2 real                  % joint velocities [rad/s]
syms ddq1 ddq2 real                % joint accelerations [rad/s^2]
syms L1 L2 positive                % link lengths [m]
syms m1 m2 positive                % link masses [kg]
syms Lc1 Lc2 positive              % distance to CoM from joint [m]
syms I1 I2 positive                % link inertias about CoM [kg*m^2]
syms g_val positive                % gravitational acceleration [m/s^2]
syms t real                        % time
```

Parameter definitions:

- q_1, q_2 -- joint angles (shoulder, elbow)
- L_1, L_2 -- link lengths (upper arm, forearm)

- m_1, m_2 -- link masses (including motors/reducers at joints)
- L_{c1}, L_{c2} -- CoM distances from proximal joint
- I_1, I_2 -- moments of inertia about each link's CoM

1.2 DH Homogeneous Transformations

The Denavit-Hartenberg convention assigns a coordinate frame to each link. For a planar 2-DOF arm moving in the vertical x-y plane with gravity acting along $-y$:

Link	a_i	α_i	d_i	θ_i
1	L_1	0	0	q_1
2	L_2	0	0	q_2

The standard DH transformation matrix for each link encodes rotation by θ_i and translation by a_i along the rotated x-axis:

```
DH = @(theta, a) [cos(theta), -sin(theta), 0, a*cos(theta);
                  sin(theta),  cos(theta), 0, a*sin(theta);
                  0,           0,          1, 0;
                  0,           0,          0, 1];
T01 = DH(q1, L1);
T12 = DH(q2, L2);
T02 = simplify(T01 * T12);
```

T_01 (base to joint 2):

```
T01
```

```
T01 =
```

$$\begin{bmatrix} \cos(q_1) & -\sin(q_1) & 0 & L_1\cos(q_1) \\ \sin(q_1) & \cos(q_1) & 0 & L_1\sin(q_1) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

T_02 (base to end-effector):

```
T02
```

```
T02 =
```

$$\begin{bmatrix} \cos(q_1+q_2) & -\sin(q_1+q_2) & 0 & L_2\cos(q_1+q_2)+L_1\cos(q_1) \\ \sin(q_1+q_2) & \cos(q_1+q_2) & 0 & L_2\sin(q_1+q_2)+L_1\sin(q_1) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1.3 End-Effector Position -- Forward Kinematics

Extract the end-effector position from the last column of T_2^0. These are the parametric forward kinematics -- every downstream quantity (Jacobian, dynamics, controller) derives from this.

End-effector x:

```
px = T02(1,4)
```

```
px =
```

$$L_2\cos(q_1+q_2)+L_1\cos(q_1)$$

End-effector y:

```
py = T02(2,4);
```

The forward kinematics in expanded form:

$$p_x = L_1\cos q_1 + L_2\cos(q_1+q_2)$$

$$p_y = L_1\sin q_1 + L_2\sin(q_1+q_2)$$

2. The Jacobian

2.1 Geometric Jacobian via `jacobian`

The velocity Jacobian $J(q)$ maps joint velocities to end-effector Cartesian velocity: $\dot{p} = J(q)\,\dot{q}$. This is the literal Jacobian matrix in the calculus sense - partial derivatives of the forward kinematics with respect to joint angles. Symbolic Math Toolbox computes it exactly.

```
p = [px; py];
q = [q1; q2];
J = jacobian(p, q);
```

Geometric Jacobian J(q):

```
J = simplify(J)
```

```
J =
```

$$\begin{bmatrix} -L_2\sin(q_1+q_2) - L_1\sin(q_1) & -L_2\sin(q_1+q_2) \\ L_2\cos(q_1+q_2) + L_1\cos(q_1) & L_2\cos(q_1+q_2) \end{bmatrix}$$

2.2 Jacobian Structure -- Term-by-Term Decomposition

Each element of the Jacobian has physical meaning. Expanding reveals which link contributes what to each velocity component.

J(1,1) -- x-velocity from shoulder rotation:

```
expand(J(1,1))
```

```
ans =
```

$$-L_1\sin(q_1) - L_2\cos(q_1)\sin(q_2) - L_2\cos(q_2)\sin(q_1)$$

J(1,2) -- x-velocity from elbow rotation:

```
expand(J(1,2))
```

```
ans =
```

$$-L_2\cos(q_1)\sin(q_2) - L_2\cos(q_2)\sin(q_1)$$

The structure shows: column 1 depends on both L_1 and L_2 (shoulder moves the entire arm), while column 2 depends only on L_2 (elbow moves only the forearm). This asymmetry is the kinematic origin of the non-diagonal inertia matrix.

2.3 Singularity Analysis

Manipulator singularities occur where $\det(J) = 0$ - the arm loses a Cartesian velocity direction. The symbolic determinant gives the exact singularity condition as a formula.

```
det_J = simplify(det(J))
```

det_J = $L_1 L_2 \sin(q_2)$

General singularity condition:

```
[singularityQ2, singularityParams, singularityConds] = solve(det_J == 0, q2, 'ReturnConditions', true)
```

singularityQ2 = πk

singularityParams = k

singularityConds = $k \in \mathbb{Z}$

Result: $\det(J) = L_1 L_2 \sin(q_2)$, so $q_2 = k\pi$ for integer k. In the usual principal range this gives $q_2 = 0$ or $q_2 = \pi$ (fully extended or folded). This is the well-known collinear-arm singularity. The symbolic form shows it depends only on the elbow angle, independent of shoulder angle and link lengths.

2.4 Velocity Mapping Validation

Substitute representative link lengths and verify that joint velocities map to the expected Cartesian velocities at a known configuration.

```
JVal = subs(J, [L1, L2], [0.5, 0.4]);
J_at_config = double(subs(JVal, [q1, q2], [pi/4, pi/3]));
dq_test = [1; 0.5];
v_ee = J_at_config * dq_test;
fprintf(['At q = [45, 60] deg, dq = [1, 0.5] rad/s:\n' ...
    '  End-effector velocity: [%.4f, %.4f] m/s\n' ...
    '  Speed: %.4f m/s\n'], ...
    v_ee(1), v_ee(2), ...
    norm(v_ee))
```

```
At q = [45, 60] deg, dq = [1, 0.5] rad/s:
  End-effector velocity: [-0.9331, 0.1983] m/s
  Speed: 0.9539 m/s
```

3. Lagrangian Dynamics

3.1 Kinetic and Potential Energy

The Lagrangian formulation derives equations of motion from energies rather than free-body diagrams. For a serial manipulator, we compute the kinetic energy of each link (translational + rotational) and the potential energy (gravity). CoM positions are needed for both.

Link 1 CoM position:

```
pc1_x = Lc1*cos(q1);
pc1_y = Lc1*sin(q1);
```

Link 2 CoM position:

```
pc2_x = L1*cos(q1) + Lc2*cos(q1 + q2);
pc2_y = L1*sin(q1) + Lc2*sin(q1 + q2);
```

CoM velocities via the chain rule (time derivatives through joint velocities):

```
dpc1_x = diff(pc1_x, q1)*dq1;
dpc1_y = diff(pc1_y, q1)*dq1;
dpc2_x = diff(pc2_x, q1)*dq1 + diff(pc2_x, q2)*dq2;
dpc2_y = diff(pc2_y, q1)*dq1 + diff(pc2_y, q2)*dq2;
```

Kinetic energy: $T = \frac{1}{2} m_i v_{ci}^2 + \frac{1}{2} I_i \dot{\theta}_i^2$

```
T1 = (1/2)*m1*(dpc1_x^2 + dpc1_y^2) + (1/2)*I1*dq1^2;
T2 = (1/2)*m2*(dpc2_x^2 + dpc2_y^2) + (1/2)*I2*(dq1 + dq2)^2;
T = simplify(T1 + T2)
```

```
T =
```

$$\frac{I_1\,\mathrm{dq_1}^2}{2} + \frac{I_2\,\mathrm{dq_1}^2}{2} + \frac{I_2\,\mathrm{dq_2}^2}{2} + \frac{L_1^2\,\mathrm{dq_1}^2\,m_2}{2} + \frac{\mathrm{Lc_1}^2\,\mathrm{dq_1}^2\,m_1}{2} + \frac{\mathrm{Lc_2}^2\,\mathrm{dq_1}^2\,m_2}{2} + \frac{\mathrm{Lc_2}^2\,\mathrm{dq_2}^2\,m_2}{2} + I_2\,\mathrm{dq_1}\,\mathrm{dq_2} + \mathrm{Lc_2}^2\,\mathrm{dc}$$

Potential energy (gravity acts in $-y$ direction):

```
V = m1*g_val*pc1_y + m2*g_val*pc2_y;
V = simplify(V)
```

V = $g_{\mathrm{val}}\, m_2\, (\mathrm{Lc}_2 \sin(q_1 + q_2) + L_1 \sin(q_1)) + \mathrm{Lc}_1\, g_{\mathrm{val}}\, m_1 \sin(q_1)$

3.2 Inertia Matrix Extraction

The kinetic energy has the quadratic form $T = \frac{1}{2}\dot{q}^T M(q)\dot{q}$. Extract the inertia matrix $M(q)$ by taking the Hessian of T with respect to $\dot{q}$ - the second partial derivatives of a quadratic form recover the matrix.

```
dq = [dq1; dq2];
```

Compute $M(q)$ by taking the Hessian of T with respect to $\dot{q}$:

```
M = simplify(jacobian(jacobian(T, dq).', dq))
```

M =

$$\begin{bmatrix} m_2 L_1^2 + 2\, m_2 \cos(q_2)\, L_1\, \mathrm{Lc}_2 + m_1\, \mathrm{Lc}_1^2 + m_2\, \mathrm{Lc}_2^2 + I_1 + I_2 & m_2\, \mathrm{Lc}_2^2 + L_1\, m_2 \cos(q_2)\, \mathrm{Lc}_2 + I_2 \\ m_2\, \mathrm{Lc}_2^2 + L_1\, m_2 \cos(q_2)\, \mathrm{Lc}_2 + I_2 & m_2\, \mathrm{Lc}_2^2 + I_2 \end{bmatrix}$$

3.3 Structure of the Inertia Matrix

The off-diagonal coupling term $M_{12} = M_{21}$ determines how acceleration at one joint creates torque at the other. Its dependence on $\cos(q_2)$ is the signature of a serial manipulator, and the reason dynamics are configuration-dependent.

M(1,1) -- effective inertia at shoulder:

```
simplify(M(1,1))
```

ans = $m_2 L_1^2 + 2\, m_2 \cos(q_2)\, L_1\, \mathrm{Lc}_2 + m_1\, \mathrm{Lc}_1^2 + m_2\, \mathrm{Lc}_2^2 + I_1 + I_2$

M(1,2) = M(2,1) -- coupling inertia:

```
simplify(M(1,2))
```

ans = $m_2\, \mathrm{Lc}_2^2 + L_1\, m_2 \cos(q_2)\, \mathrm{Lc}_2 + I_2$

M(2,2) -- effective inertia at elbow:

```
simplify(M(2,2))
```

ans = $m_2 \, \mathrm{Lc}_2^{\,2} + I_2$

M_{11} is the full-arm inertia (depends on both links), M_{22} is the forearm-only inertia (constant), and M_{12} contains the configuration-dependent coupling that makes the dynamics nonlinear.

3.4 Christoffel Symbols and Coriolis/Centrifugal Matrix

The Coriolis and centrifugal terms $C(q,\dot{q})\dot{q}$ arise from the velocity-dependence of kinetic energy. Compute them via Christoffel symbols of the first kind: $c_{ijk} = \frac{1}{2}\left(\frac{\partial M_{kj}}{\partial q_i} + \frac{\partial M_{ki}}{\partial q_j} - \frac{\partial M_{ij}}{\partial q_k}\right)$

```
n_joints = 2;
C_matrix = sym(zeros(n_joints));
for k_idx = 1:n_joints
    for j_idx = 1:n_joints
        c_kj = sym(0);
        for i_idx = 1:n_joints
            christoffel = (1/2)*(diff(M(k_idx,j_idx), q(i_idx)) + ...
                                 diff(M(k_idx,i_idx), q(j_idx)) - ...
                                 diff(M(i_idx,j_idx), q(k_idx)));
            c_kj = c_kj + christoffel * dq(i_idx);
        end
        C_matrix(k_idx, j_idx) = c_kj;
    end
end
```

Coriolis/centrifugal matrix C(q, dq):

```
C_matrix = simplify(C_matrix)
```

C_matrix =

$$\begin{bmatrix} -L_1\,\mathrm{Lc}_2\,\mathrm{dq}_2\,m_2\sin(q_2) & -L_1\,\mathrm{Lc}_2\,m_2\sin(q_2)\,(\mathrm{dq}_1+\mathrm{dq}_2) \\ L_1\,\mathrm{Lc}_2\,\mathrm{dq}_1\,m_2\sin(q_2) & 0 \end{bmatrix}$$

3.5 Gravity Torque Vector

The gravity torque $G(q) = \partial V/\partial q$ -- exact partial derivatives via `diff`.

```
G_vec = [diff(V, q1); diff(V, q2)];
G_vec = simplify(G_vec)
```

```
G_vec =
```

$$\begin{bmatrix} g_{\mathrm{val}}\,m_2\,(\mathrm{Lc}_2\cos(q_1+q_2)+L_1\cos(q_1))+\mathrm{Lc}_1\,g_{\mathrm{val}}\,m_1\cos(q_1) \\ \mathrm{Lc}_2\,g_{\mathrm{val}}\,m_2\cos(q_1+q_2) \end{bmatrix}$$

The gravity torque on joint 1 depends on both links (it supports the entire arm); the torque on joint 2 depends only on the forearm.

3.6 Complete Equations of Motion

The manipulator equations of motion in standard form:

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau$$

where τ is the vector of applied joint torques. This decomposition into M, C, and G is exactly what a computed-torque controller needs.

```
syms tau1 tau2 real
ddq = [ddq1; ddq2];
tau_vec = [tau1; tau2];
EOM = M*ddq + C_matrix*dq + G_vec - tau_vec;
disp('Equations of motion M*ddq + C*dq + G = tau:')
```

```
Equations of motion M*ddq + C*dq + G = tau:
```

Joint 1:

```
simplify(EOM(1))
```

```
ans =
```

$$\mathrm{ddq}_2\,\left(m_2\,\mathrm{Lc}_2{}^2+L_1\,m_2\cos(q_2)\,\mathrm{Lc}_2+I_2\right)-\tau_1+\mathrm{ddq}_1\,\left(m_2\,L_1{}^2+2\,m_2\cos(q_2)\,L_1\,\mathrm{Lc}_2+m_1\,\mathrm{Lc}_1{}^2+m_2\,\mathrm{Lc}_2{}^2+I\right.$$

Joint 2:

```
simplify(EOM(2))
```

ans = $L_1\,\mathrm{Lc}_2\,m_2\sin(q_2)\,\mathrm{dq}_1^2 - \tau_2 + \mathrm{ddq}_1\,\left(m_2\,\mathrm{Lc}_2^2 + L_1\,m_2\cos(q_2)\,\mathrm{Lc}_2 + I_2\right) + \mathrm{ddq}_2\,\left(m_2\,\mathrm{Lc}_2^2 + I_2\right) + \mathrm{Lc}_2\,g_{\mathrm{val}}\,m_2\,c$

4. Analytical Sensitivities

4.1 Torque Sensitivity to Link Parameters

How does the required shoulder torque change with forearm mass? This exact partial derivative reveals the coupling coefficient, which is critical for payload capacity analysis and motor sizing.

```
tau1_expr = simplify(M(1,:)*ddq + C_matrix(1,:)*dq + G_vec(1));
dtau1_dm2 = simplify(diff(tau1_expr, m2))
```

dtau1_dm2 = $\mathrm{ddq}_1\,\left(L_1^2 + 2\cos(q_2)\,L_1\,\mathrm{Lc}_2 + \mathrm{Lc}_2^2\right) + \mathrm{ddq}_2\,\left(\mathrm{Lc}_2^2 + L_1\cos(q_2)\,\mathrm{Lc}_2\right) + g_{\mathrm{val}}\,\left(\mathrm{Lc}_2\cos(q_1+q_2) + L_1\cos(q_1)\right)$

This sensitivity is a formula in $(q_1, q_2, \dot{q}, \ddot{q})$ -- it tells you exactly how much additional shoulder torque each kilogram of payload demands at every configuration. The velocity-dependent part comes from the Coriolis/centrifugal term and matters whenever the arm is moving.

```
velocitySensitivity = simplify(diff(C_matrix(1,:)*dq, m2))
```

velocitySensitivity = $-L_1\,\mathrm{Lc}_2\,\mathrm{dq}_2\sin(q_2)\,\left(2\,\mathrm{dq}_1 + \mathrm{dq}_2\right)$

4.2 Kinematic Singularity vs. Dynamic Conditioning

The Jacobian singularity is a kinematic property: it makes the Cartesian velocity map rank deficient. The inertia matrix $M(q)$ is a separate dynamic object; for positive masses and inertias it remains positive definite even when the arm is kinematically singular.

```
manipulabilitySquared = simplify(det_J^2)
```

manipulabilitySquared = $L_1^2\,L_2^2\sin(q_2)^2$

```
det_M = simplify(det(M))
```

det_M = $-L_1^2\,\mathrm{Lc}_2^2\,m_2^2\cos(q_2)^2 + L_1^2\,\mathrm{Lc}_2^2\,m_2^2 + I_2\,L_1^2\,m_2 + m_1\,\mathrm{Lc}_1^2\,\mathrm{Lc}_2^2\,m_2 + I_2\,m_1\,\mathrm{Lc}_1^2 + I_1\,\mathrm{Lc}_2^2\,m_2 + I_1\,I_2$

```
det_M_positive_form = L1^2*Lc2^2*m2^2*sin(q2)^2 + I2*L1^2*m2 + ...
    m1*Lc1^2*Lc2^2*m2 + I2*m1*Lc1^2 + I1*Lc2^2*m2 + I1*I2;
det_M_check = simplify(det_M - det_M_positive_form)
```

det_M_check = 0

The squared manipulability goes to zero at $q_2 = k\pi$, while the determinant of M can be rewritten as a sum of positive physical terms. This is enough to show the key distinction: use $\det(J)$ or manipulability for kinematic singularities; use $M(q)$ for dynamic conditioning.

5. Computed-Torque Controller

5.1 Control Law

The computed-torque (inverse dynamics) controller cancels the nonlinear dynamics and imposes linear second-order error dynamics:

$$\tau = M(q)\left(\ddot{q}_d + K_d(\dot{q}_d - \dot{q}) + K_p(q_d - q)\right) + C(q,\dot{q})\dot{q} + G(q)$$

With this control law, the closed-loop error $e = q_d - q$ satisfies $\ddot{e} + K_d\dot{e} + K_p e = 0$ -- a linear ODE with exactly placed poles, regardless of configuration or velocity.

```
syms Kp Kd positive                    % PD gains (scalar, same for both joints)
syms q1d q2d dq1d dq2d ddq1d ddq2d real  % desired trajectory
qd = [q1d; q2d];
dqd = [dq1d; dq2d];
ddqd = [ddq1d; ddq2d];
e = qd - q;
de = dqd - dq;
v = ddqd + Kd*de + Kp*e;              % auxiliary input (desired acceleration + PD correction)
tau_ct = M*v + C_matrix*dq + G_vec;   % computed-torque control law
```

Computed-torque control law (joint 1):

```
simplify(tau_ct(1))
```

ans = $g_{\mathrm{val}}\, m_2\, (\mathrm{Lc}_2 \cos(q_1 + q_2) + L_1 \cos(q_1)) - (\mathrm{Kd}\ (\mathrm{dq}_1 - \mathrm{dq1d}) - \mathrm{ddq1d} + \mathrm{Kp}\ (q_1 - \mathrm{q1d}))\ \left(m_2 {L_1}^2 + 2\, m_2 \cos(\right.$

5.2 Closed-Loop Error Dynamics via `dsolve`

The error dynamics are linear by construction. Solve the characteristic equation to verify exponential convergence. Choose gains for critically damped response with natural frequency $\omega_n = 10$ rad/s and damping ratio $\zeta = 1$.

```
syms e_sym(t)
omega_n = 10;    % rad/s
zeta = 1;        % critically damped
Kp_val = omega_n^2;
Kd_val = 2*zeta*omega_n;
error_ode = diff(e_sym, t, 2) + Kd_val*diff(e_sym, t) + Kp_val*e_sym == 0;
e_sol = dsolve(error_ode, e_sym(0) == 1, subs(diff(e_sym,t), t, 0) == 0);
```

Error dynamics solution (critically damped):

```
simplify(e_sol)
```

ans = $\mathrm{e}^{-10\,t}\ (10\,t + 1)$

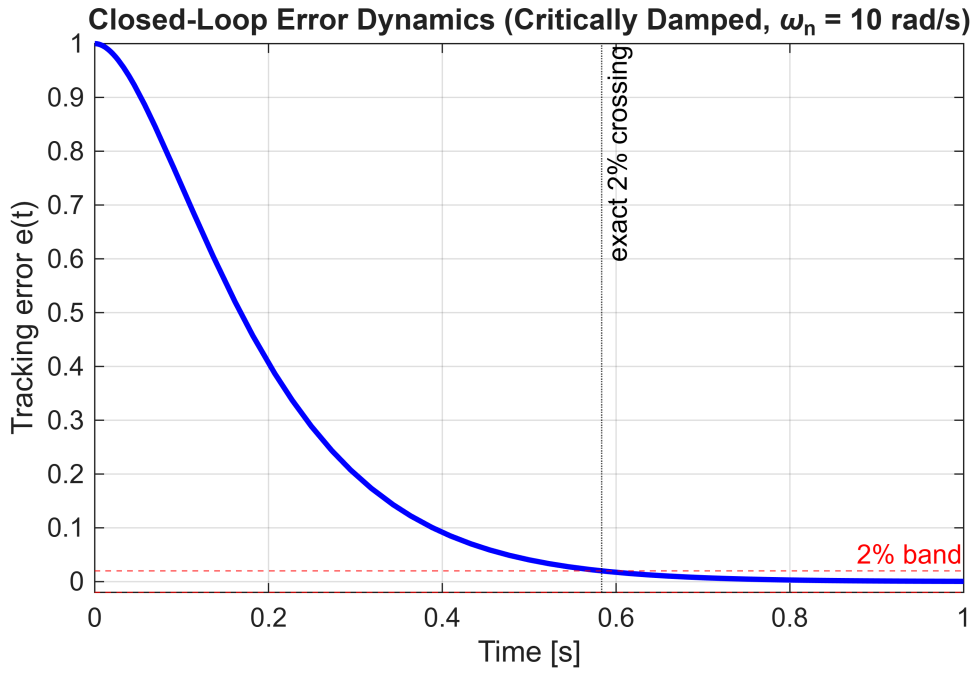
```
settlingTimeApprox = 4/(zeta*omega_n);
settlingTimeExactVal = double(vpasolve(e_sol == 0.02, t, [0, 1]));
fprintf(['Gains: Kp = %.0f, Kd = %.0f\n' ...
    '  Rule-of-thumb 2%% settling time: %.2f s\n' ...
    '  Exact 2%% crossing for this response: %.3f s\n'], ...
    Kp_val, Kd_val, ...
    settlingTimeApprox, settlingTimeExactVal)
```

```
Gains: Kp = 100, Kd = 20
  Rule-of-thumb 2% settling time: 0.40 s
  Exact 2% crossing for this response: 0.583 s
```

5.3 Error Convergence Visualization

```
fplot(e_sol, [0 1], 'b-', 'LineWidth', 2)
xlabel('Time [s]')
```

```
ylabel('Tracking error e(t)')
title('Closed-Loop Error Dynamics (Critically Damped, \omega_n = 10 rad/s)')
grid on
yline(0.02, 'r--', '2% band');
yline(-0.02, 'r--');
xline(settlingTimeExactVal, 'k:', 'exact 2% crossing');
```



6. Code Generation for Downstream Workflows

6.1 Generate Inertia Matrix Function

Convert the symbolic $M(q)$ to an optimized MATLAB® function. The generated code evaluates the inertia matrix at any configuration without re-deriving. It is suitable for controller prototyping and for use inside a MATLAB Function block.

```
matlabFunction(M, 'File', 'armInertiaMatrix', ...
    'Vars', {q1, q2, I1, I2, L1, Lc1, Lc2, m1, m2}, ...
    'Outputs', {'M'}, 'Optimize', true);
```

```
fprintf('\n--- Generated armInertiaMatrix.m ---\n')
```

```
--- Generated armInertiaMatrix.m ---
```

```
type armInertiaMatrix.m
```

```
function M = armInertiaMatrix(q1,q2,I1,I2,L1,Lc1,Lc2,m1,m2)
%armInertiaMatrix
%    M = armInertiaMatrix(Q1,Q2,I1,I2,L1,Lc1,Lc2,M1,M2)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    10-Aug-2026 14:21:32

t2 = cos(q2);
t3 = Lc2.^2;
t4 = m2.*t3;
t5 = L1.*Lc2.*m2.*t2;
t6 = I2+t4+t5;
M = reshape([I1+t5+t6+L1.^2.*m2+Lc1.^2.*m1,t6,t6,I2+t4],[2,2]);
end
```

6.2 Generate Coriolis Matrix Function

```
matlabFunction(C_matrix, 'File', 'armCoriolisMatrix', ...
    'Vars', {q2, dq1, dq2, L1, Lc2, m2}, ...
    'Outputs', {'C'}, 'Optimize', true);
fprintf('\n--- Generated armCoriolisMatrix.m ---\n')
```

```
--- Generated armCoriolisMatrix.m ---
```

```
type armCoriolisMatrix.m
```

```
function C = armCoriolisMatrix(q2,dq1,dq2,L1,Lc2,m2)
%armCoriolisMatrix
%    C = armCoriolisMatrix(Q2,DQ1,DQ2,L1,Lc2,M2)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    10-Aug-2026 14:21:33

t2 = sin(q2);
C = reshape([-L1.*Lc2.*dq2.*m2.*t2,L1.*Lc2.*dq1.*m2.*t2,-L1.*Lc2.*m2.*t2.*(dq1+dq2),0.0],[2,2]);
end
```

6.3 Generate Gravity Torque Function

```
matlabFunction(G_vec, 'File', 'armGravityTorque', ...
    'Vars', {q1, q2, L1, Lc1, Lc2, g_val, m1, m2}, ...
    'Outputs', {'G'}, 'Optimize', true);
fprintf('\n--- Generated armGravityTorque.m ---\n')
```

```
--- Generated armGravityTorque.m ---
```

```
type armGravityTorque.m
```

```
function G = armGravityTorque(q1,q2,L1,Lc1,Lc2,g_val,m1,m2)
%armGravityTorque
%    G = armGravityTorque(Q1,Q2,L1,Lc1,Lc2,G_VAL,M1,M2)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    10-Aug-2026 14:21:33

t2 = cos(q1);
t3 = q1+q2;
t4 = cos(t3);
G = [g_val.*m2.*(L1.*t2+Lc2.*t4)+Lc1.*g_val.*m1.*t2;Lc2.*g_val.*m2.*t4];
end
```

6.4 Generate Jacobian Function

```
matlabFunction(J, 'File', 'armJacobian', ...
    'Vars', {q1, q2, L1, L2}, ...
    'Outputs', {'J'}, 'Optimize', true);
fprintf('\n--- Generated armJacobian.m ---\n')
```

```
--- Generated armJacobian.m ---
```

```
type armJacobian.m
```

```
function J = armJacobian(q1,q2,L1,L2)
%armJacobian
%    J = armJacobian(Q1,Q2,L1,L2)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    10-Aug-2026 14:21:33

t2 = q1+q2;
```

```
t3 = cos(t2);
t4 = sin(t2);
t5 = L2.*t3;
t6 = L2.*t4;
t7 = -t6;
J = reshape([t7-L1.*sin(q1),t5+L1.*cos(q1),t7,t5],[2,2]);
end
```

6.5 Generate Simscape Equation Syntax

`simscapeEquation` converts symbolic expressions to Simscape language equation syntax. This is the useful Symbolic Math Toolbox bridge: it gives the equations you would place inside a custom Simscape component, without turning this example into a Simscape component-authoring tutorial.

```
syms theta1 theta2 omega1 omega2 alpha1 alpha2 g real
tauDynamicsSimscape = subs(M*ddq + C_matrix*dq + G_vec, ...
    [q1, q2, dq1, dq2, ddq1, ddq2, g_val], ...
    [theta1, theta2, omega1, omega2, alpha1, alpha2, g]);
simscapeTorqueEquations = [string(simscapeEquation(tau1, tauDynamicsSimscape(1)));
                           string(simscapeEquation(tau2, tauDynamicsSimscape(2)))];
simscapeTorqueEquations = replace(simscapeTorqueEquations, ...
    ["alpha1", "alpha2"], ["omega1.der", "omega2.der"]);
simscapeTorqueEquations
```

```
simscapeTorqueEquations = 2×1 string
"tau1 == omega2.der*(I2+Lc2^2*m2+L1*Lc2*m2*cos(theta2))+omega1.der*(I1+I2+L1^2*m ···
"tau2 == omega1.der*(I2+Lc2^2*m2+L1*Lc2*m2*cos(theta2))+omega2.der*(I2+Lc2^2*m2)
```

7. Simulation: Computed-Torque Tracking

7.1 Robot Parameters

Representative 2-DOF planar elbow manipulator in the vertical plane:

- Link 1: 0.5 m, 5.0 kg (includes joint 2 motor), CoM at 0.25 m
- Link 2: 0.4 m, 3.0 kg (forearm + end-effector), CoM at 0.20 m
- Inertias: uniform rod approximation $I = \frac{1}{12}mL^2$

```
L1_val  = 0.5;                          % m
L2_val  = 0.4;                          % m
m1_val  = 5.0;                          % kg
m2_val  = 3.0;                          % kg
Lc1_val = 0.25;                         % m
Lc2_val = 0.20;                         % m
I1_val  = (1/12)*m1_val*L1_val^2;       % kg*m^2
I2_val  = (1/12)*m2_val*L2_val^2;       % kg*m^2
g_num   = 9.81;                         % m/s^2
fprintf(['Link 1: L=%.2f m, m=%.1f kg, I=%.4f kg*m^2\n' ...
    'Link 2: L=%.2f m, m=%.1f kg, I=%.4f kg*m^2\n'], ...
    L1_val, m1_val, I1_val, ...
    L2_val, m2_val, I2_val)
```

```
Link 1: L=0.50 m, m=5.0 kg, I=0.1042 kg*m^2
Link 2: L=0.40 m, m=3.0 kg, I=0.0400 kg*m^2
```

7.2 Desired Trajectory: Circular End-Effector Path

Command the end-effector to trace a circle of radius 0.1 m centered at (0.5, 0.3) m. Inverse kinematics converts the Cartesian path to joint-space commands. The trajectory runs for 2 seconds at constant angular rate.

```
dt = 0.001;                             % 1 kHz control rate
t_sim = 0:dt:2;
N_sim = length(t_sim);
r_circle = 0.1;                         % m
cx = 0.5; cy = 0.3;                     % circle center
omega_traj = pi;                        % rad/s (one half-circle in 1 s)
px_traj = cx + r_circle*cos(omega_traj*t_sim);
py_traj = cy + r_circle*sin(omega_traj*t_sim);
dpx_traj = -r_circle*omega_traj*sin(omega_traj*t_sim);
dpy_traj =  r_circle*omega_traj*cos(omega_traj*t_sim);
ddpx_traj = -r_circle*omega_traj^2*cos(omega_traj*t_sim);
ddpy_traj = -r_circle*omega_traj^2*sin(omega_traj*t_sim);
```

Inverse kinematics for desired joint trajectories (elbow-up):

```
cq2_traj = (px_traj.^2 + py_traj.^2 - L1_val^2 - L2_val^2) / (2*L1_val*L2_val);
```

```
cq2_traj = max(min(cq2_traj, 1), -1);
sq2_traj = sqrt(1 - cq2_traj.^2);
q2d_traj = atan2(sq2_traj, cq2_traj);
q1d_traj = atan2(py_traj, px_traj) - atan2(L2_val*sin(q2d_traj), L1_val + L2_val*cos(q2d_traj));
```

Joint velocities and accelerations via Jacobian inversion. The acceleration uses $\ddot{q}_d = J^{-1}(\ddot{p}_d - \dot{J}\dot{q}_d)$, so the feedforward term has no finite-difference lag at the first sample.

```
Jdot = simplify(diff(J, q1)*dq1 + diff(J, q2)*dq2);
JdotFcn = matlabFunction(Jdot, 'Vars', {q1, q2, dq1, dq2, L1, L2});
dq_traj = zeros(2, N_sim);
ddq_traj = zeros(2, N_sim);
for k = 1:N_sim
    J_k = armJacobian(q1d_traj(k), q2d_traj(k), L1_val, L2_val);
    cartesianVelocity_k = [dpx_traj(k); dpy_traj(k)];
    cartesianAcceleration_k = [ddpx_traj(k); ddpy_traj(k)];
    dq_traj(:,k) = J_k \ cartesianVelocity_k;
    Jdot_k = JdotFcn(q1d_traj(k), q2d_traj(k), dq_traj(1,k), dq_traj(2,k), L1_val, L2_val);
    ddq_traj(:,k) = J_k \ (cartesianAcceleration_k - Jdot_k*dq_traj(:,k));
end
```

7.3 Closed-Loop Self-Consistency Simulation

Simulate the computed-torque controller tracking the circular path. The controller and the simple rigid-body plant both use the generated symbolic functions (M, C, G), so this checks generated-function consistency and controller wiring. It is not an independent physical validation. Initial condition: start at the first trajectory point with the desired initial velocity. The state update uses a second-order kinematic position step instead of forward Euler.

```
x_state = [q1d_traj(1); dq_traj(1,1); q2d_traj(1); dq_traj(2,1)];
q_hist = zeros(2, N_sim);
tau_hist = zeros(2, N_sim);
e_hist = zeros(2, N_sim);
for k = 1:N_sim
    q_k = [x_state(1); x_state(3)];
    dq_k = [x_state(2); x_state(4)];
    qd_k = [q1d_traj(k); q2d_traj(k)];
    dqd_k = dq_traj(:,k);
    ddqd_k = ddq_traj(:,k);
```

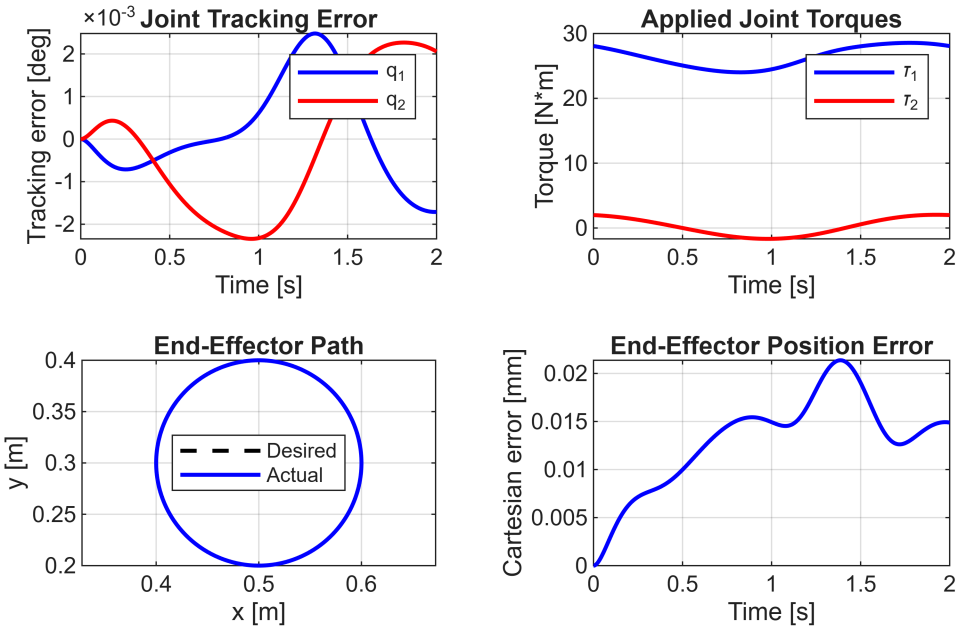
```
    e_k = qd_k - q_k;
    de_k = dqd_k - dq_k;
    v_k = ddqd_k + Kd_val*de_k + Kp_val*e_k;
    M_k = armInertiaMatrix(q_k(1), q_k(2), I1_val, I2_val, L1_val, Lc1_val, Lc2_val, m1_val, m2_val);
    C_k = armCoriolisMatrix(q_k(2), dq_k(1), dq_k(2), L1_val, Lc2_val, m2_val);
    G_k = armGravityTorque(q_k(1), q_k(2), L1_val, Lc1_val, Lc2_val, g_num, m1_val, m2_val);
    tau_k = M_k*v_k + C_k*dq_k + G_k;
    q_hist(:,k) = q_k;
    tau_hist(:,k) = tau_k;
    e_hist(:,k) = e_k;
    ddq_actual = M_k \ (tau_k - C_k*dq_k - G_k);
    dq_next = dq_k + dt*ddq_actual;
    q_next = q_k + dt*dq_k + 0.5*dt^2*ddq_actual;
    x_state = [q_next(1); dq_next(1); q_next(2); dq_next(2)];
end
fprintf(['Peak tracking error: %.4f rad (%.3f deg)\n' ...
    'Peak torque (joint 1): %.2f N*m\n' ...
    'Peak torque (joint 2): %.2f N*m\n'], ...
    max(abs(e_hist(:))), rad2deg(max(abs(e_hist(:)))), ...
    max(abs(tau_hist(1,:))), ...
    max(abs(tau_hist(2,:))))
```

```
Peak tracking error: 0.0000 rad (0.002 deg)
Peak torque (joint 1): 28.56 N*m
Peak torque (joint 2): 2.04 N*m
```

7.4 Tracking Results

```
tiledlayout(2,2)
nexttile
plot(t_sim, rad2deg(e_hist(1,:)), 'b-', t_sim, rad2deg(e_hist(2,:)), 'r-', 'LineWidth', 1.5)
xlabel('Time [s]'); ylabel('Tracking error [deg]')
title('Joint Tracking Error')
legend('q_1', 'q_2', 'Location', 'northeast'); grid on
nexttile
plot(t_sim, tau_hist(1,:), 'b-', t_sim, tau_hist(2,:), 'r-', 'LineWidth', 1.5)
xlabel('Time [s]'); ylabel('Torque [N*m]')
title('Applied Joint Torques')
```

```
legend('\tau_1', '\tau_2', 'Location', 'northeast'); grid on
nexttile
px_actual = L1_val*cos(q_hist(1,:)) + L2_val*cos(q_hist(1,:) + q_hist(2,:));
py_actual = L1_val*sin(q_hist(1,:)) + L2_val*sin(q_hist(1,:) + q_hist(2,:));
plot(px_traj, py_traj, 'k--', px_actual, py_actual, 'b-', 'LineWidth', 1.5)
xlabel('x [m]'); ylabel('y [m]')
title('End-Effector Path')
legend('Desired', 'Actual', 'Location', 'best'); grid on
axis equal
nexttile
cart_error = sqrt((px_traj - px_actual).^2 + (py_traj - py_actual).^2)*1000;
plot(t_sim, cart_error, 'b-', 'LineWidth', 1.5)
xlabel('Time [s]'); ylabel('Cartesian error [mm]')
title('End-Effector Position Error')
grid on
```



8. Downstream Workflow Connections

8.1 Robotics System Toolbox: Validate Jacobian Against rigidBodyTree

Build the same arm as a `rigidBodyTree` using the DH parameters from Phase 1, then compare the toolbox's `geometricJacobian` against our symbolic result at a test configuration. This cross-checks the kinematic convention against an independent Robotics System Toolbox implementation.

```
robot = rigidBodyTree('DataFormat', 'row');
body1 = rigidBody('link1');
jnt1 = rigidBodyJoint('q1', 'revolute');
setFixedTransform(jnt1, [L1_val 0 0 0], 'dh');
body1.Joint = jnt1;
body1.Mass = m1_val;
body1.CenterOfMass = [Lc1_val 0 0];
body1.Inertia = [0 0 I1_val 0 0 0];
addBody(robot, body1, 'base');
body2 = rigidBody('link2');
jnt2 = rigidBodyJoint('q2', 'revolute');
setFixedTransform(jnt2, [L2_val 0 0 0], 'dh');
body2.Joint = jnt2;
body2.Mass = m2_val;
body2.CenterOfMass = [Lc2_val 0 0];
body2.Inertia = [0 0 I2_val 0 0 0];
addBody(robot, body2, 'link1');
q_test = [pi/4, pi/3];
J_symbolic = armJacobian(q_test(1), q_test(2), L1_val, L2_val);
J_rst = geometricJacobian(robot, q_test, 'link2');
J_rst_linear = J_rst(4:5, :);         % extract linear velocity rows (x, y)
```

Jacobian validation at q = [45, 60] deg:

Symbolic J:

```
J_symbolic
```

```
J_symbolic = 2×2
   -0.7399   -0.3864
    0.2500   -0.1035
```

Robotics System Toolbox J (linear, xy):

```
J_rst_linear
```

```
J_rst_linear = 2×2
   -0.7399   -0.3864
    0.2500   -0.1035
```

```
fprintf('Max absolute difference: %.2e\n', max(abs(J_symbolic(:) - J_rst_linear(:))))
```

```
Max absolute difference: 1.67e-16
```

8.2 Simscape™ and Simulink®

The generated MATLAB functions are the direct path into Simulink MATLAB Function blocks. The Simscape equation strings from Section 6.5 are a lighter bridge for users who want to author a custom Simscape component later.

8.3 Parameter Sensitivity from Symbolic Torque

Use the exact torque sensitivity from Section 4.1 to determine how payload mass propagates into actuator requirements. This is the symbolic math payoff: a formula for torque margin, not a simulation sweep.

```
dtau1_dm2_Val = subs(dtau1_dm2, [L1, Lc2, g_val], [L1_val, Lc2_val, g_num]);
```

Evaluate at a zero-velocity, fully extended posture with specified joint accelerations. The velocity sensitivity term is zero for this point, but it contributes during dynamic motion.

```
dtau1_Val = double(subs(dtau1_dm2_Val, [q1, q2, dq1, dq2, ddq1, ddq2], ...
    [0, 0, 0, 0, 10, 5]));
fprintf(['Shoulder torque sensitivity at worst case: %.2f N*m per kg of payload\n' ...
    '  For 2 kg payload increase: +%.1f N*m required torque margin\n'], ...
    dtau1_Val, ...
    2*dtau1_Val)
```

```
Shoulder torque sensitivity at worst case: 12.47 N*m per kg of payload
  For 2 kg payload increase: +24.9 N*m required torque margin
```